

AI-STUDYBUDDY

Master Technical Documentation Package

Item	Details
Team ID / Size	SWTID-2026-7347 4 members
Team Leader	Himesh Roshan M J
Team Members	Harish M, Jagatheeshwaran K, Karthikeyan R
Tech Stack	Node.js, Express.js, MongoDB (Mongoose), JWT, bcryptjs, Multer, Google Gemini (gemini-2.5-flash)
Document Index	1 Problem Statement 2 SRS 3 Architecture 4 Database 5 REST API 6 Task Allocation 7 Test Plan 8 Deployment 9 Demo Script 10 Final Report

1. Problem Statement & Scope

1.1 Executive Summary

Students in higher education work with large volumes of lecture notes, textbooks and supplementary material that are fragmented and dense. Turning that raw material into summaries, practice questions and a realistic study timetable takes hours of manual effort that could be spent actually learning. AI-STUDYBUDDY is a RESTful backend that ingests a student's own study material and uses a Large Language Model (Google Gemini) to automatically produce summaries, quizzes, flashcards and personalised study plans.

1.2 Problem Statement

- Time-intensive content synthesis:** extracting core principles from dense text by hand is slow and error-prone.
- Lack of automated self-assessment:** students rarely have practice quizzes built directly from their own syllabus material.
- Inflexible planning:** converting learning objectives into a realistic day-by-day schedule is difficult without a structured tool.

1.3 Project Objectives

- Secure backend:** build with Node.js, Express.js, MongoDB and JSON Web Tokens (JWT).
- Document ingestion:** accept text files with Multer and store metadata and raw text in MongoDB.
- AI generation:** integrate Gemini (gemini-2.5-flash) for summaries, flashcards, quizzes and study plans.
- Data protection:** secure all user data and AI endpoints with Bearer-token headers and bcryptjs hashing.

1.4 Target Users & Benefits

Primary users are university and college students who want faster revision. The system also suits tutors who want to generate quick quizzes from their own notes. Because every material belongs to one authenticated user, each person's study content stays private.

1.5 Scope & Boundaries

In Scope	Out of Scope
JWT registration and login	Native mobile client applications
Single-file upload (multipart/form-data)	Direct payment gateway integration
MongoDB storage of metadata and raw text	Multi-file or folder uploads
AI endpoints: summary, flashcards, quiz, study plan	Real-time collaboration between users
Centralised error handling and async middleware	

1.6 Assumptions, Constraints & Risks

Type	Description
Assumption	Users supply plain-text (.txt) notes and have a stable internet connection.
Constraint	AI quality and speed depend on the Google Gemini API and its rate limits.
Risk	Very long documents may exceed model input limits; mitigation is to chunk or trim content (future work).
Risk	Leaked secrets (JWT secret, API key); mitigation is to keep them in .env, never in source control.

2. Software Requirement Specification (SRS)

2.1 Purpose & Overview

This SRS defines what AI-STUDYBUDDY must do and the quality attributes it must meet. The system is split into four modules, each owned by one team member, so requirements are grouped by module for clear accountability.

2.2 Functional Requirements

ID	Module (Owner)	Requirement	Priority
FR-1.1	Auth (Harish M)	Register users with name, email and password.	High
FR-1.2	Auth	Hash passwords with bcryptjs before saving to MongoDB.	High
FR-1.3	Auth	Issue a signed JWT after successful credential validation.	High
FR-1.4	Auth	Protect routes using Authorization: Bearer <TOKEN>.	High
FR-2.1	Ingestion (Jagatheeshwaran K)	Accept .txt uploads through Multer middleware.	High
FR-2.2	Ingestion	Persist file content and metadata linked to the user's ID.	High
FR-2.3	Ingestion	Let the user retrieve all materials they own.	Medium
FR-3.1	AI Core (Karthikeyan R)	Generate concise bullet-point summaries via Gemini.	High
FR-3.2	AI Core	Generate multiple-choice quizzes with options, answers and explanations.	High
FR-3.3	AI Core	Generate question/answer flashcards for key terms.	Medium
FR-3.4	AI Core	Generate structured daily study plans from a topic and number of days.	Medium
FR-4.1	Gateway (Himesh Roshan M J)	Handle cross-origin requests with the cors middleware.	Medium
FR-4.2	Gateway	Catch route errors and return standardised JSON responses.	High

2.3 Non-Functional Requirements

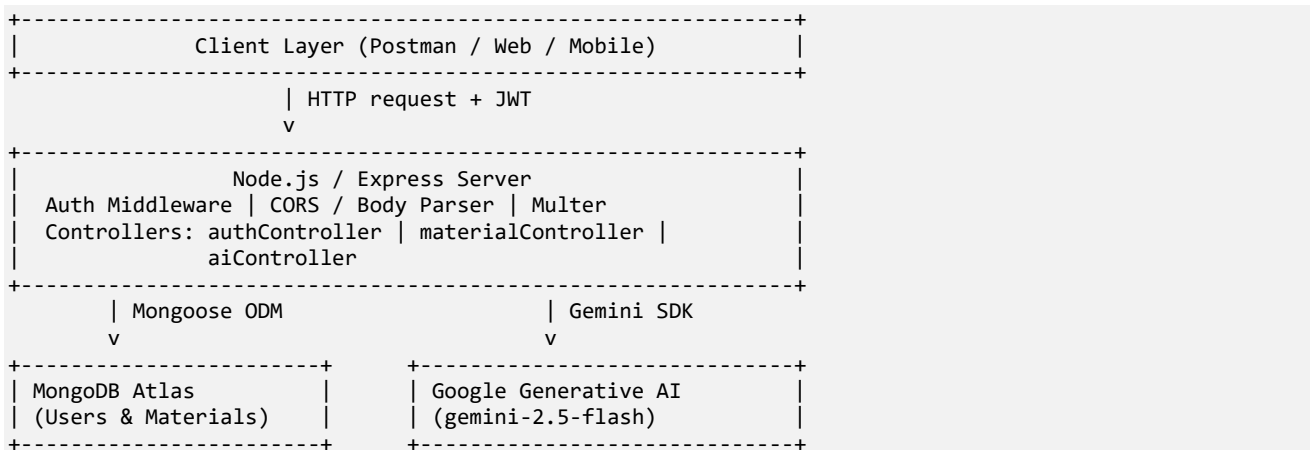
Category	Requirement and explanation
Security (NFR-1)	Passwords hashed with salt factor 10, so identical passwords produce different hashes. JWTs expire after 24 hours, limiting the damage of a stolen token.
Performance (NFR-2)	AI responses should return in under 3 seconds under standard network latency; this is a target and depends on Gemini response time.
Maintainability (NFR-3)	Modular Express design separating Controllers, Models, Routes and Services, so each part can be changed or tested independently.

2.4 Acceptance Criteria

- A new user can register, log in and receive a token.
- An authenticated user can upload a .txt file and see it listed under their account.
- Each of the four AI endpoints returns valid output for an owned material.
- Requests without a valid token are rejected with HTTP 401.

3. System Architecture & Design

3.1 High-Level Architecture



3.2 Layer Explanation

- **Client layer:** any HTTP client (Postman today, a React app later) that sends JSON or multipart requests.
- **Middleware layer:** CORS allows cross-origin calls, the body parser reads JSON, Multer reads uploaded files, and authMiddleware.js checks the JWT.
- **Controller layer:** holds the business logic for authentication, materials and AI generation.
- **Data layer:** Mongoose models map JavaScript objects to MongoDB collections.
- **AI layer:** geminiService.js wraps the Google Generative AI SDK so controllers never talk to Gemini directly.

3.3 Request Data Flow

- **Step 1:** the client sends Authorization: Bearer <JWT> with the request payload.
- **Step 2:** authMiddleware.js verifies the token with process.env.JWT_SECRET and attaches the user to the request.
- **Step 3:** Express routes forward the call to the matching controller action.
- **Step 4:** the controller reads or writes MongoDB through Mongoose, or calls gemini-2.5-flash through the SDK.
- **Step 5:** the result, or a standardised error, is returned as JSON.

3.4 Project Structure & Design Decisions

```
AI-STUDYBUDDY/
server.js          (app entry, middleware, DB connection)
controllers/       authController, materialController, aiController
middleware/        authMiddleware
models/            User, Material
routes/            authRoutes, materialRoutes, aiRoutes
services/          geminiService
.env               PORT, MONGO_URI, JWT_SECRET, GEMINI_API_KEY
```

Key choices: JWT keeps the API stateless so it scales without server-side sessions; uploaded files are handled in memory and their text is saved in MongoDB, so no file storage is needed; the Gemini call sits in a service layer so the model can be swapped without touching controllers.

4. Database Schema & ER Documentation

AI-STUDYBUDDY uses MongoDB with two collections. Mongoose enforces structure and validation in the application layer.

4.1 Users Collection (models/User.js)

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  createdAt: { type: Date, default: Date.now }
});
```

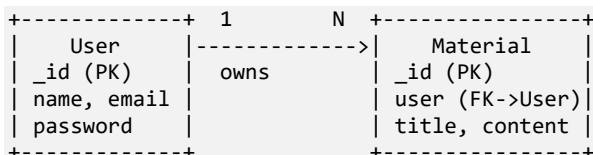
Field	Type	Purpose
name	String	Display name of the student.
email	String (unique)	Login identifier; the unique index blocks duplicate registrations (TC-02).
password	String	bcrypt hash only; the plain password is never stored.
createdAt	Date	Account creation timestamp, set automatically.

4.2 Materials Collection (models/Material.js)

```
const materialSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  title: { type: String, required: true },
  content: { type: String, required: true },
  fileType: { type: String, default: 'txt' },
  uploadedAt: { type: Date, default: Date.now }
});
```

Field	Type	Purpose
user	ObjectId (ref User)	Owner of the material; used to restrict every query to the logged-in user.
title	String	Name of the uploaded document.
content	String	Full raw text, sent to Gemini when generating AI output.
fileType	String	Defaults to 'txt'; allows other formats (PDF) in future.
uploadedAt	Date	Upload timestamp, set automatically.

4.3 Entity Relationship



One User owns many Materials (1 : N). Material.user references User._id. Querying with the authenticated user's ID guarantees one student can never read another's materials.

5. REST API Specification

Base URL: <http://localhost:5000>. All bodies are JSON except the upload, which is multipart/form-data. Private routes need the header Authorization: Bearer <TOKEN>.

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Create an account; returns 201 with token and user object.
POST	/api/auth/login	Public	Validate credentials; returns a JWT valid for 24 hours.
POST	/api/materials/upload	Private	Upload a .txt file in form field 'file'; returns 201 with material metadata.
GET	/api/materials	Private	List every material owned by the authenticated user.
POST	/api/ai/summarize	Private	Body: materialId. Returns a bullet-point summary.
POST	/api/ai/quiz	Private	Body: materialId, numQuestions. Returns MCQs with options, answers, explanations.
POST	/api/ai/flashcards	Private	Body: materialId. Returns question/answer pairs for key terms.
POST	/api/ai/study-plan	Private	Body: topic, days. Returns a day-by-day study timeline.

Sample: Register

```
POST /api/auth/register
{ "name": "Himesh Roshan", "email": "himesh@example.com", "password": "Password123" }

201 Created
{ "token": "eyJhbGciOiJIUzI1NiIs... ",
  "user": { "id": "66f1a...", "name": "Himesh Roshan", "email": "himesh@example.com" } }
```

Sample: Summarize

```
POST /api/ai/summarize
{ "materialId": "<MATERIAL_ID>" }

{ "summary": "- Express is a minimal Node.js framework.\n- JWTs provide stateless auth." }
```

Sample: Study Plan Request

```
POST /api/ai/study-plan
{ "topic": "Node.js REST APIs", "days": 3 }
```

Status Codes

Code	Meaning in this API
201	Resource created (registration, upload).
400	Bad request, such as a duplicate email ("User already exists").
401	Missing or invalid Bearer token.
500	Unexpected server or AI-service error, returned as standardised JSON.

6. Module Responsibility & Task Allocation

The work was divided by module so that each developer owned a clear set of files, and the team leader integrated the pieces and tested the full flow.

ID	Module	Developer	Deliverables	Status
MOD-01	User Authentication & JWT	Harish M	authController.js, authMiddleware.js, User.js	Completed
MOD-02	Material Ingestion & In-Memory Uploads	Jagatheeshwaran K	materialController.js, materialRoutes.js, Material.js	Completed
MOD-03	AI Core Engine & Gemini Integration	Karthikeyan R	aiController.js, geminiService.js, aiRoutes.js	Completed
MOD-04	Leadership, Architecture & Integration	Himesh Roshan M J	server.js, .env configuration, API testing and integration	Completed

6.1 Detailed Responsibilities

- **Harish M (Authentication):** registration and login logic, bcryptjs hashing, JWT signing, and the middleware that protects private routes.
- **Jagatheeshwaran K (Ingestion):** Multer setup for in-memory uploads, reading file text, saving materials against the user ID, and the material listing route.
- **Karthikeyan R (AI Core):** designing prompts for each generation type, the Gemini service wrapper, and parsing model output into structured JSON.
- **Himesh Roshan M J (Leadership):** overall architecture, server bootstrap, CORS, centralised error handling, environment config, integration of all modules and end-to-end testing in Postman.

6.2 Collaboration & Integration Workflow

- **Shared contract:** the team agreed on the API specification (Document 5) and the schemas (Document 4) before coding.
- **Independent development:** each member built and tested their module in isolation.
- **Integration:** the team leader merged routes into server.js and ran the end-to-end scenarios in Document 7.
- **Review:** every module was checked against its functional requirements in Document 2.

7. Master Test Plan & Test Cases

7.1 Test Strategy

Testing is black-box and API-level, performed with Postman against a running server and a test MongoDB database. It covers authentication, file ingestion and AI generation, and checks both success paths and expected failures.

7.2 Executed Test Cases

ID	Module	Scenario	Input / Action	Expected Result	Status
TC-01	Auth	User registration	POST /api/auth/register, valid email and password	HTTP 201 + JWT token	PASS
TC-02	Auth	Duplicate email	Register with an existing email	HTTP 400 'User already exists'	PASS
TC-03	Auth	Protected route, no token	GET /api/materials with no Auth header	HTTP 401 Unauthorized	PASS
TC-04	Upload	File ingestion	POST /api/materials/upload with sample_notes.txt	HTTP 201 + material metadata	PASS
TC-05	AI Core	Summarization	POST /api/ai/summarize, valid materialId	Summary text from Gemini	PASS
TC-06	AI Core	Quiz generation	POST /api/ai/quiz, valid materialId	Structured JSON quiz array	PASS

7.3 Recommended Additional Test Cases

These cases extend coverage and have not been recorded as executed; run them before final submission.

ID	Module	Scenario	Input / Action	Expected Result	Status
TC-07	Auth	Wrong password	POST /api/auth/login with an incorrect password	Rejected, no token issued	Planned
TC-08	Auth	Invalid token	Call a private route with a tampered token	HTTP 401 Unauthorized	Planned
TC-09	Upload	No file attached	POST /api/materials/upload with empty form	Clear error message	Planned
TC-10	AI Core	Unknown materialId	POST /api/ai/flashcards with a non-existent ID	Not-found or error JSON	Planned
TC-11	AI Core	Study plan	POST /api/ai/study-plan with topic and days: 3	Three-day structured plan	Planned
TC-12	Security	Cross-user access	User B requests User A's materialId	Access denied	Planned

8. Deployment, Setup & Environment Guide

8.1 Prerequisites

- Node.js v18.x or higher and npm.
- MongoDB Atlas cluster, or a local MongoDB instance.
- Google Gemini API key from Google AI Studio.
- Postman (or similar) for testing the API.

8.2 Environment Variables (.env)

```
PORT=5000
MONGO_URI=mongodb+srv://<username>:<password>@cluster0.mongodb.net/ai-
studybuddy?retryWrites=true&w=majority
JWT_SECRET=<your_strong_secret>
GEMINI_API_KEY=<your_gemini_api_key>
```

Variable	Explanation
PORT	Port the Express server listens on (5000 by default).
MONGO_URI	Connection string; replace username and password and keep the database name ai-studybuddy.
JWT_SECRET	Long random string used to sign and verify tokens; changing it invalidates existing tokens.
GEMINI_API_KEY	Authenticates calls to the Gemini API; never share or commit it.

Keep `.env` in `.gitignore` so secrets are never pushed to a repository.

8.3 Installation & Running

```
# Clone and enter the project directory
cd AI-STUDYBUDDY
```

```
# Install dependencies
npm install
```

```
# Start the backend server
npm start
```

On success the console shows the server running on port 5000 and a MongoDB connection message.

8.4 Troubleshooting

Symptom	Likely cause and fix
MongoDB connection error	Check MONGO_URI credentials and whitelist your IP in Atlas Network Access.
401 on private routes	Token missing, expired or sent without the 'Bearer ' prefix.
AI endpoints fail	GEMINI_API_KEY missing or invalid, or the API quota is exhausted.
Port already in use	Change PORT in <code>.env</code> or stop the other process.

9. Video Demo Script & Screen Recording Guide

Presenter: Himesh Roshan M J. Suggested total length: about 6 to 8 minutes. Use Postman for requests and VS Code for the server.

Layer	Time	Actions and talking points
1. Setup	1 min	Open the VS Code terminal and run npm start. Show the server on port 5000 and the 'MongoDB connected' message. Explain the stack in one sentence.
2. Authentication	1.5 min	In Postman send POST /api/auth/login. Point out the JWT in the response and copy it. Mention bcrypt hashing and 24-hour expiry.
3. Material Ingestion	1.5 min	Add the Bearer token in the Authorization tab. Run POST /api/materials/upload with the test file and copy the returned _id. Run GET /api/materials to show it is stored.
4. AI Generation	3 min	Trigger /api/ai/summarize and show the bulleted summary. Trigger /api/ai/quiz and show options and explanations. Trigger /api/ai/study-plan and display the 3-day timeline.
5. Wrap-up	30 sec	Recap the features and mention future scope: React frontend, PDF/OCR, analytics.

Recording Tips

- **Preparation:** register a demo user and prepare sample_notes.txt before recording.
- **Screen:** use 1080p, a large font in VS Code and Postman, and close unrelated tabs.
- **Privacy:** hide the .env file and never show real API keys or database passwords on screen.
- **Audio:** record in a quiet room and narrate each request before sending it.

10. Final Comprehensive Project Report

10.1 Abstract

AI-STUDYBUDDY is an AI-powered learning assistant backend built on the MERN stack. It uses Google Gemini models to parse academic materials and produce bulleted summaries, practice quizzes, flashcards and personalised study routines. The system enforces strict JWT authentication, uses Mongoose models for reliable MongoDB data management, and connects to the Gemini API through a dedicated service layer.

10.2 Methodology

- **Requirements:** the problem was analysed and converted into functional and non-functional requirements.
- **Design:** a layered architecture, two database schemas and a REST contract were defined up front.
- **Implementation:** four modules were built in parallel by individual owners, then integrated.
- **Verification:** API-level test cases were run in Postman for authentication, upload and AI generation.

10.3 Outcomes

- **Security:** passwords are hashed, tokens expire, and all private routes require a valid Bearer token.
- **Productivity:** a student can go from raw notes to a summary, quiz, flashcards and study plan with a few requests.
- **Quality:** modular code and standardised errors make the system easy to extend and debug.

10.4 Challenges & Limitations

- **Input format:** only plain-text (.txt) files are supported today.
- **AI dependency:** response time and output quality depend on the external Gemini service.
- **Document size:** very large texts may exceed model limits and would need chunking.
- **Interface:** there is no graphical frontend yet; the API is tested through Postman.

10.5 Conclusion

The backend core of AI-STUDYBUDDY is fully implemented, tested against the main scenarios, and ready for deployment. All primary objectives, namely user security, material upload handling and multi-mode AI generation, have been fulfilled.

10.6 Future Scope

- **Frontend UI:** a responsive React.js web interface.
- **PDF & OCR ingestion:** extend the Multer pipeline to PDFs and images using Tesseract OCR.
- **Analytics dashboard:** track quiz scores and progress over time.
- **Smarter AI:** chunking for long documents, spaced-repetition scheduling and personalised difficulty.